



Taller Práctico: Memoria y Archivos

Malak Sanchez

Monitorias Sistemas Operativos

18 de septiembre de 2026

Introducción

El siguiente taller tiene como objetivo integrar los conceptos fundamentales vistos en el Capítulo 0: paso de parámetros por línea de comandos, gestión de memoria dinámica y manipulación de archivos. Los problemas planteados requieren un análisis detallado de la estructura de datos y una gestión responsable de los recursos del sistema.

Ejercicio 1

Diseñe un programa que genere en memoria dinámica un tablero de caracteres alternados. El programa debe recibir por línea de comandos dos enteros positivos N y M , y dos caracteres A y B .

Ejemplo de ejecución:

```
$ ./tablero 4 5 X 0
```

Ejemplo de salida:

```
X 0 X 0 X
0 X 0 X 0
X 0 X 0 X
0 X 0 X 0
```

Requerimientos:

1. El programa debe validar que se pasen los 4 argumentos necesarios.
2. Debe reservar memoria dinámica para una matriz de caracteres (`char **`) de tamaño $N \times M$.
3. Rellene cada posición con A o B según la paridad de la suma de sus índices.
4. Implemente una función `print_matrix` y una función `free_matrix` para gestionar la limpieza.
5. El programa debe imprimir la matriz en pantalla.
6. La matriz creada debe ser dinámica, es decir, debe poder aceptar cualquier valor de N y M e imprimir correctamente el patrón.

Ejercicio 2

Diseñe una herramienta que procese un archivo de texto con registros de envíos. La primera fila del archivo contiene un valor n que indica la cantidad de registros. Cada línea representa un envío con el formato:

```
<ID_ENVIO> <DISTANCIA_KM> <PESO_KG>
```

Ejemplo de comando:

```
$ ./envios input.txt 15
```

Ejemplo de salida esperada (Peso superior a 15 kg):

```
1024 38.5 16.0
1025 12.0 20.5
1026 92.0 18.0
```

Requerimientos:

1. Debe leer el archivo y almacenar los datos correctamente.
2. Debe usar `fscanf(file, "%d%f%f", &id, &distancia, &peso)` para leer los datos del archivo con el formato especificado.
3. Filtre y muestre solo los envíos cuyo peso sea superior al umbral pasado por parámetro.
4. Cuente la cantidad de envíos que superan el umbral e imprima el resultado.

Ejercicio 3

Implemente un programa que reciba como parámetro un archivo `input.in` y genere una codificación por tramos de caracteres (RLE - Run-Length Encoding).

Ejemplo de comando:

```
$ ./rle_encode input.in
```

Ejemplo visual de funcionamiento:

```
Entrada (en archivo): XXYYYYZZZZ  
Salida (consola): X2Y4Z4
```

Requerimientos:

1. Cargue el contenido del archivo pasado como argumentos del main.
2. El programa debe validar que el archivo de entrada no esté vacío antes de proceder.
3. Almacene la cadena codificada en un *buffer*, usando el formato carácter seguido de cantidad.
4. Imprima en pantalla el “Ratio de Compresión” (Tamaño Original / Tamaño Comprimido).
5. Libere correctamente la memoria dinámica utilizada.

Criterios de Evaluación

- **Gestión de Memoria:** Uso correcto de `malloc/free` y ausencia de fugas.
- **Robustez:** El programa no debe fallar si el archivo no existe o los argumentos son insuficientes.
- **Claridad:** Código bien estructurado y documentado en lugares clave.